RTOS SPI
开发指南

版本号: 2.3

发布日期: 2025.07.18

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.07.16	AWA1636	初始版本
1.1	2021.04.14	AWA1636	2.2 小节，增加 sys_config.fex 文件的配置方法
1.2	2022.10.28	AWA1979	文档增加 R128 IC 相关内容
1.3	2023.03.21	AWA1979	文档增加 MR527 IC 相关内容
1.4	2023.03.29	AWA1979	文档增加 XR875 IC 相关内容
1.5	2023.05.29	AWA1979	文档增加 AI985 IC 相关内容
2.0	2023.06.12	AWA1979	1.RTOS SPI 代码重构 2. 支持 slave 模式及测试用例使用说明
2.1	2023.08.02	AWA1979	增加 Bit-Aligned 模式支持
2.2	2023.11.16	AWA1979	文档增加 T113-S3 IC 相关内容
2.3	2024.08.14	AWA2155	文档增加 MR536 IC 相关内容
2.3	2025.07.18	AWA1669	文档增加 F135 IC 相关内容

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 1

1.1 文档简介 1

1.2 目标读者 1

1.3 适用范围 1

1.4 文档约定 2

1.4.1 标志说明 2

1.4.2 地址与数据描述方法约定 2

1.4.3 数值单位约定 2

1.5 相关术语介绍 3

1.5.1 硬件术语 3

1.5.2 软件术语 3

2 模块介绍 5

2.1 模块功能介绍 5

2.2 模块配置介绍 5

2.2.1 sys_config 文件引脚配置说明 5

2.2.2 平台头文件资源配置说明 6

2.2.3 menuconfig 配置说明 6

2.3 源码结构介绍 7

3 模块接口说明 8

3.1 重要结构体及宏定义 8

3.1.1 SPI 模式功能选择 8

3.1.2 SPI 控制器模式配置 8

3.1.3 SPI 控制器片选模式 9

3.1.4 SPI 控制器采样模式 9

3.1.5 SPI 控制器配置结构体 10

3.1.6 SPI 传输结构体 10

3.2 对外提供的 API 接口 10

3.2.1 hal_spi_init 10

3.2.2 hal_spi_deinit 11

3.2.3 hal_spi_write 11

3.2.4 hal_spi_read 11

3.2.5 hal_spi_write_then_read 12

3.2.6 hal_spi_xfer 12

3.2.7 hal_spi_slave_abort 12

4 模块使用范例 13

4.1 Master 模式使用及测试范例 13

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 BIT 模式使用及测试范例 13
4.3 Slave 模式使用及测试范例 14
4.3.1 Slave 模式使用范例 14
4.3.2 Slave 模式测试范例 14

5 调试方法 15
5.1 调试工具 15

6 FAQ 16
6.1 多笔数据连续发送时，如何让 CS 脚保持使能 16

广州星翼电子科技有限公司zdyzlinux

1 前言

1.1 文档简介

本文档介绍了 RTOS 中 SPI 驱动的接口及使用方法，为 SPI 使用者提供参考。

1.2 目标读者

SPI 驱动层/应用层开发/使用/维护人员。

1.3 适用范围

表 1-1: 适用产品列表

产品名称	内核版本	驱动文件
F133	Melis	hal_spi.c
V833	Melis	hal_spi.c
F135	Melis	hal_spi.c
R328	FreeRTOS	hal_spi.c
R128	FreeRTOS	hal_spi.c
MR527	FreeRTOS	hal_spi.c
XR875	FreeRTOS	hal_spi.c
AI985	FreeRTOS	hal_spi.c
T113	FreeRTOS	hal_spi.c
MR536	FreeRTOS	hal_spi.c

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1.4 文档约定

1.4.1 标志说明

注意

- **提醒操作中应注意的事项。不当的操作可能会损坏器件，影响可靠性、降低性能等。**

说明

为准确理解文中指令、正确实施操作而提供的补充或强调信息。

技巧

一些容易忽视的小功能、技巧。了解这些功能或技巧能帮助解决特定问题或者节省操作时间。

1.4.2 地址与数据描述方法约定

本文档在描述地址、数据时遵循如下约定：

表 1-2: 地址与数据描述方法约定

符号	例子	说明
0x	0x0200，0x79	地址或数据以 16 进制表示。
0b	0b010，0b00 000 111	数据采用二进制表示 (寄存器描述除外)。
X	00X，XX1	数据描述中，X 代表 0 或 1。 例如，00X 代表 000 或 001；XX1 代表 001，011，101 或 111。

1.4.3 数值单位约定

本文档在描述数据容量（如 NAND 容量）时，单位词头代表的是 1024 的倍数；描述频率、数据速率等时则代表的是 1000 的倍数。具体如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 1-3: 数值单位约定

类型	符号	对应数值
数据容量（如 NAND 容量）	1 K	1024
	1 M	1 048 576
	1 G	1 073 741 824
频率，数据速率等	1 k	1000
	1 M	1 000 000
	1 G	1 000 000 000

1.5 相关术语介绍

1.5.1 硬件术语

表 1-4: 硬件术语

术语	解释说明
SUNXI	指 Allwinner 的一系列 SoC 硬件平台
SPI	Serial Peripheral Interface，同步串行外设接口
SPI Master	SPI 主模式
SPI Slave	SPI 从模式
SPI Device	挂在 SPI 总线上的设备

1.5.2 软件术语

版权所有 © 珠海全志科技股份有限公司。保留一切权利

表 1-5: 软件术语

术语	解释说明
sys_config.fex	格式化的文本文件，是全志对 DDR/Uboot 等相关配置文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

SPI 是一种全双工同步串行接口，可以工作在 Master 模式和 Slave 模式，SPI 主要有以下特点：

- 全双工同步串行接口
- Master/Slave 模式可配置
- 支持最大 100MHz 时钟频率
- 支持 SPI Mode0/1/2/3
- 支持多片选
- 片选和时钟的极性和相位可配置
- 支持 CPU(中断) 或 DMA 模式传输
- 支持 TX/RX FIFO 缓存
- 支持 Standard Single/Dual/Quad 数据模式（Slave 模式不支持 Dual/Quad）
- 支持 BIT 模式，用于 3Wire 场景，支持可编程 0~32bits 帧长度（仅支持 Master 模式，且不支持 DMA 和 FIFO 功能）
- 支持 DBI 模式，用于显示设备场景，用于传输视频数据（具体支持平台见芯片手册）

2.2 模块配置介绍

目前有两种方法进行引脚配置：sys_config 和平台头文件。

这两种方法选其中一种使用即可，在没有 sys_config 文件配置时，默认使用平台头文件。当检测到有 sys_config 文件并有相关关键字时，自动切换为该方法。

2.2.1 sys_config 文件引脚配置说明

引脚配置在 source/project/方案/configs/sys_confi.fex 中存放。

```
[spi0]
spi0_sclk       = port:PC02<2><0><2><default>
spi0_cs         = port:PC03<2><1><2><default>
spi0_mosi       = port:PC04<2><0><2><default>
spi0_miso       = port:PC05<2><0><2><default>
spi0_wp         = port:PC06<2><0><2><default>
spi0_hold       = port:PC07<2><0><2><default>
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

说明

引脚说明：port: 端口 < 复用功能 >< 上下拉 >< 驱动能力 >< 输出值 >

2.2.2 平台头文件资源配置说明

引脚配置在 rtos-hal/hal/source/spi/platform/spi_{CHIP}.h（CHIP 是软件研发代号）。

```
static struct sunxi_spi_params_t g_sunxi_spi_params[] = {
  /* SPI0 */
  {  .port = 0,
     .reg_base = SUNXI_SPI0_PBASE, .irq_num = SUNXI_IRQ_SPI0, .gpio_num = 6,
     .pclk_pll_type = HAL_SUNXI_CCU, .pclk_pll_id = CLK_PLL_PERI0_300M,
     .pclk_hosc_type = HAL_SUNXI_FIXED_CCU, .pclk_hosc_id = CLK_SRC_HOSC24M,
     .bus_type = HAL_SUNXI_CCU, .bus_id = CLK_BUS_SPI0,
     .mclk_type = HAL_SUNXI_CCU, .mclk_id = CLK_SPI0,
     .reset_type = HAL_SUNXI_RESET, .reset_id = RST_BUS_SPI0,
     .gpio_clk = GPIOC(12), .gpio_mosi = GPIOC(2), .gpio_miso = GPIOC(4),
     .gpio_cs0 = GPIOC(3), .gpio_wp = GPIOC(15), .gpio_hold = GPIOC(16),
     .mux = 4, .driv_level = GPIO_DRIVING_LEVEL2,
  #if defined(SUNXI_USED_DMA0)
     .drq_tx = DRQDST_SPI0_TX, .drq_rx = DRQSRC_SPI0_RX,
  #endif
     .rx_fifosize = 128, .tx_fifosize = 64, .dma_force_fixed = false,
  },
  ......
};
```

2.2.3 menuconfig 配置说明

配置路径如下：

```
Kernel Setup  --->
  Drivers Setup  --->
    SoC HAL Drivers  --->
      SPI Devices  --->
```

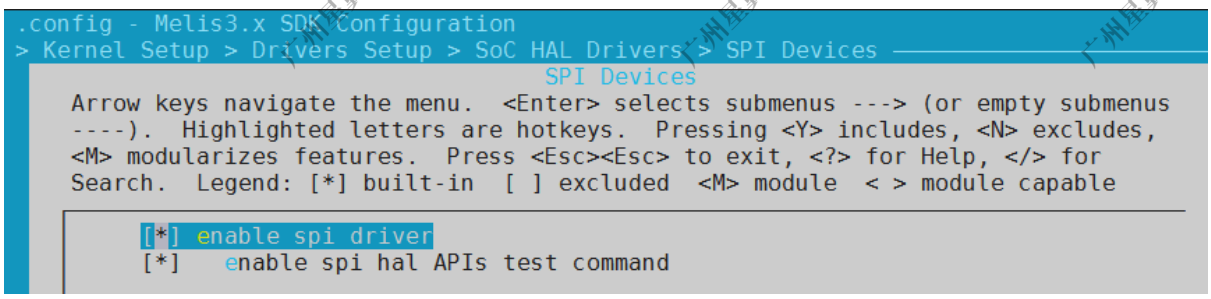

图 2-1: SPI menuconfig

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.3 源码结构介绍

SPI 模块代码结构如下所示：

```
rtos-hal/
|--include/hal/sunxi_hal_spi.h        // hal层数据结构和API接口相关头文件
|--hal/source/spi/platform_spi.h      // hal层平台相关头文件
|--hal/source/spi/platform/spi_xxx.h  // hal层平台信息相关头文件
|--hal/source/spi/common_spi.h        // hal层控制器寄存器相关头文件
|--hal/source/spi/hal_spi.c           // hal层接口驱动代码
|
|--hal/test/spi/test_spi.c            // hal层接口测试代码
|--hal/test/spi/spi_slave_driver.c    // hal层slave模式驱动代码
|--hal/test/spi/test_spi_slave.c      // hal层slave模式测试代码
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

需要包含头文件：

```
#include <hal/sunxi_hal_spi.h>
```

3.1 重要结构体及宏定义

3.1.1 SPI 模式功能选择

```
#define SPI_CPHA      BIT(0)  /* clock phase */
#define SPI_CPOL      BIT(1)  /* clock polarity */

#define SPI_MODE_0    (0|0)
#define SPI_MODE_1    (0|SPI_CPHA)
#define SPI_MODE_2    (SPI_CPOL|0)
#define SPI_MODE_3    (SPI_CPOL|SPI_CPHA)

#define SPI_CS_HIGH    BIT(2)  /* chipselect active high? */
#define SPI_LSB_FIRST  BIT(3)  /* per-word bits-on-wire */
#define SPI_3WIRE      BIT(4)  /* SI/SO signals shared */
#define SPI_LOOP       BIT(5)  /* loopback mode */
#define SPI_NO_CS      BIT(6)  /* 1 dev/bus, no chipselect */
#define SPI_READY      BIT(7)  /* slave pulls low to pause */
#define SPI_TX_DUAL    BIT(8)  /* transmit with 2 wires */
#define SPI_TX_QUAD    BIT(9)  /* transmit with 4 wires */
#define SPI_RX_DUAL    BIT(10) /* receive with 2 wires */
#define SPI_RX_QUAD    BIT(11) /* receive with 4 wires */
```

- SPI_MODE_0/1/2/3：设置 SPI 的传输模式。
- SPI_CS_HIGH：设置 CS 片选是否为高电平有效。
- SPI_LSB_FIRST：设置发送顺序是低位在前。
- SPI_3WIRE：设置 SPI 工作在 3 线模式下，及 MOSI 即用作输入也用作输入，实现半双工通信

3.1.2 SPI 控制器模式配置

```
typedef enum
{
    HAL_SPI_BUS_MASTER = 0,
    HAL_SPI_BUS_SLAVE = 1,
    HAL_SPI_BUS_BIT = 2,
} hal_spi_master_bus_mode_t;
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- HAL_SPI_BUS_MASTER：处于 Master 模式，外接 SPI Device。
- HAL_SPI_BUS_SLAVE：处于 Slave 模式，被其他 Master 访问。
- HAL_SPI_BUS_BIT：处于 BIT 模式，使用 3Wire 方式进行数据传输。

3.1.3 SPI 控制器片选模式

```
typedef enum
{
  HAL_SPI_CS_AUTO = 0,
  HAL_SPI_CS_SOFT = 1,
} hal_spi_master_cs_mode_t;
```

- HAL_SPI_CS_AUTO：硬件自动控制，不需要驱动或软件介入。
- HAL_SPI_CS_SOFT：软件手动控制，由驱动完成相关操作。

3.1.4 SPI 控制器采样模式

```
typedef enum
{
  SUNXI_SPI_SAMP_MODE_OLD = 0,
  SUNXI_SPI_SAMP_MODE_NEW = 1,
} hal_spi_master_bus_sample_mode_t;
```

- SUNXI_SPI_SAMP_MODE_OLD：粗调模式，共有 3 档可调
- SUNXI_SPI_SAMP_MODE_NEW：细调模式，共有 7 档可调

说明

粗调模式为驱动根据时钟频率自动识别，不需要额外配置。

```
typedef enum
{
  SUNXI_SPI_SAMP_DELAY_CYCLE_0_0 = 0,
  SUNXI_SPI_SAMP_DELAY_CYCLE_0_5 = 1,
  SUNXI_SPI_SAMP_DELAY_CYCLE_1_0 = 2,
  SUNXI_SPI_SAMP_DELAY_CYCLE_1_5 = 3,
  SUNXI_SPI_SAMP_DELAY_CYCLE_2_0 = 4,
  SUNXI_SPI_SAMP_DELAY_CYCLE_2_5 = 5,
  SUNXI_SPI_SAMP_DELAY_CYCLE_3_0 = 6,
} hal_spi_master_spi_sample_mode_t;
```

- SUNXI_SPI_SAMP_DELAY_CYCLE：采样延时调节挡位选择。

说明

当采样模式处于细调时，才会使用到该参数。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.1.5 SPI 控制器配置结构体

```
typedef struct
{
    hal_spi_master_bus_mode_t bus_mode; // SPI控制器配置
    hal_spi_master_cs_mode_t cs_mode;   // SPI控制器片选模式
    hal_spi_master_bus_sample_mode_t bus_sample_mode;   // SPI控制器采样模式 - 粗调
    hal_spi_master_spi_sample_mode_t spi_sample_mode;   // SPI控制器采样模式 - 细调
    uint32_t spi_sample_delay;  // SPI控制器细调采样延时
    uint8_t chipselect; /* SPI slave device selection */
    uint32_t clock_frequency; /* SPI master clock frequency setting */
    uint32_t mode;  // SPI模式/功能选择
    bool sip;
    bool flash;
} hal_spi_master_config_t;
```

3.1.6 SPI 传输结构体

```
typedef struct
{
    const uint8_t *tx_buf;  /* Data buffer to send */
    uint32_t tx_len;        /* The total number of bytes to send */
    uint32_t tx_single_len; /* The number of bytes to send in single mode */
    uint8_t *rx_buf;        /* Received data buffer, */
    uint32_t rx_len;        /* The valid number of bytes received */
    uint8_t tx_nbits : 3;   /* Data buffer to send in nbits mode */
    uint8_t rx_nbits : 3;   /* Data buffer to received in nbits mode */
    uint8_t dummy_byte;     /* Flash send dummy byte, default 0*/
#define SPI_NBITS_SINGLE    0x01 /* 1bit transfer */
#define SPI_NBITS_DUAL      0x02 /* 2bits transfer */
#define SPI_NBITS_QUAD      0x04 /* 4bits transfer */
    uint8_t bits_per_word;  /* transfer bit_per_word */
} hal_spi_master_transfer_t;
```

3.2 对外提供的 API 接口

3.2.1 hal_spi_init

- 原型: hal_spi_master_status_t hal_spi_init(int port, hal_spi_master_config_t *cfg)
- 作用：SPI 模块初始化，主要申请中断、pinctrl 初始化、clk 初始化、SPI 模块，包括 SPI 总线最大传输速率、片选模式等等。
- 参数：
  - port:SPI 端口号
  - cfg: 配置信息
- 返回：
  - 0: 成功

版权所有 © 珠海全志科技股份有限公司。保留一切权利

  - 负数: 失败

3.2.2 hal_spi_deinit

- 原型: hal_spi_master_status_t hal_spi_deinit(int port)
- 作用：SPI 模块去初始化。
- 参数：
  - port:SPI 端口号
- 返回：
  - 0: 成功
  - 负数: 失败

3.2.3 hal_spi_write

- 原型: hal_spi_master_status_t hal_spi_write(int port, const uint8_t *buf, uint32_t size)
- 作用：发送数据，调 hal_spi_xfer 接口。
- 参数：
  - port:SPI 端口号
  - buf: 发送数据
  - size: 发送数据大小
- 返回：
  - 0: 成功
  - 负数: 失败

3.2.4 hal_spi_read

- 原型: hal_spi_master_status_t hal_spi_read(int port, uint8_t *buf, uint32_t size)
- 作用：接收数据，调 hal_spi_xfer 接口。
- 参数：
  - port:SPI 端口号
  - buf: 接收数据
  - size: 接收数据大小
- 返回：
  - 0: 成功
  - 负数: 失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.5 hal_spi_write_then_read

- 原型: hal_spi_master_status_t hal_spi_write_then_read(int port, void *tbuf, uint32_t tlen, void *rbuf, uint32_t rlen)
- 作用：先写数据然后紧接着进行读操作，调 hal_spi_xfer 接口。
- 参数：
  - port:SPI 端口号
  - tbuf: 发送数据
  - tlen: 发送数据大小
  - rbuf: 接收数据
  - rlen: 接收数据大小
- 返回：
  - 0: 成功
  - 负数: 失败

3.2.6 hal_spi_xfer

- 原型: hal_spi_master_status_t hal_spi_xfer(int port, hal_spi_master_transfer_t *t, int num)
- 作用：发送或接收数据。
- 参数：
  - port:SPI 端口号
  - t: 指向传输包头的指针
  - num: 传输包的个数
- 返回：
  - 0: 成功
  - 负数: 失败

3.2.7 hal_spi_slave_abort

- 原型: hal_spi_master_status_t hal_spi_slave_abort(int port)
- 作用：终止 slave 模式传输。
- 参数：
  - port:SPI 端口号
- 返回：
  - 0: 成功
  - 负数: 失败

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块使用范例

4.1 Master 模式使用及测试范例

```
hal_spi_master_config_t cfg = { 0 };
cfg.bus_mode = HAL_SPI_BUS_MASTER;
cfg.cs_mode = HAL_SPI_CS_AUTO;
cfg.clock_frequency = 10000000;
cfg.chipselect = 0;
cfg.mode = SPI_MODE_0;
cfg.sip = 0;
cfg.flash = 0;
hal_spi_init(port, &cfg);
```

详细请参考测试用例源码（hal/test/spi/test_spi.c）。

4.2 BIT 模式使用及测试范例

```
hal_spi_master_config_t cfg = { 0 };
cfg.bus_mode = HAL_SPI_BUS_BIT;
......
hal_spi_init(port, &cfg);

hal_spi_master_transfer_t tr = {
    .tx_buf = &buf,
    .tx_len = 1,
    .rx_buf = NULL,
    .rx_len = 0,
    .bits_per_word = 8,
};
hal_spi_xfer(port, tr, 1);
```

详细请参考测试用例源码（hal/test/spi/test_spi.c）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3 Slave 模式使用及测试范例

4.3.1 Slave 模式使用范例

```
hal_spi_master_config_t cfg = { 0 };
cfg.bus_mode = HAL_SPI_BUS_SLAVE;
cfg.clock_frequency = 10000000;
cfg.chipselect = 0;
cfg.mode = SPI_MODE_0;
hal_spi_init(port, &cfg);
```

详细请参考测试用例源码（hal/test/spi/spi_slave_driver.c）。

4.3.2 Slave 模式测试范例

详细请参考测试用例源码（hal/test/spi/test_spi_slave.c）。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 调试方法

使用 hal_log_info/warn/err/debug 等调试打印方法，可以根据需求修改系统配置，达到不同的打印等级。

```
#define SPI_INFO(sspi, fmt, arg...) hal_log_info("hal-sspi %08lx.sspi%d: " fmt, sspi->base, sspi->bus_num, ##arg)
#define SPI_WARN(sspi, fmt, arg...) hal_log_warn("hal-sspi %08lx.sspi%d: " fmt, sspi->base, sspi->bus_num, ##arg)
#define SPI_ERR(sspi, fmt, arg...)  hal_log_err("hal-sspi %08lx.sspi%d: " fmt, sspi->base, sspi->bus_num, ##arg)
#define SPI_DBG(sspi, fmt, arg...)  hal_log_debug("hal-sspi %08lx.sspi%d: " fmt, sspi->base, sspi->bus_num, ##arg)
```

使用 SPI_DATA_LEVEL/SPI_DUMPREG_LEVEL 宏开关，可以在传输的过程中打印收发数据 buffer 及控制器寄存器值。

5.1 调试工具

如果在 menuconfig 中将 spi test 配置打开，那么可以在 RTOS 系统中运行相应的 SPI 调试命令。

测试命令	解释说明
hal_spidev_test	测试 SPI Single 模式，可以配置端口，频率，数据量，循环次数等参数 hal_spidev_test -D <port> -s <freq> -S <size> -l <loop> [-v]
hal_spi_slave_driver	Slave 模式驱动，需要先再 slave 设备端跑起该驱动，才能在 master 端进行相应测试 hal_spi_slave_driver probe <port> <freq> hal_spi_slave_driver remove <port> hal_spi_slave_driver abort <port> hal_spi_slave_driver dump <port> <addr> <size>
hal_spi_slave_test	Slave 模式测试命令，该命令需要在 master 端运行 hal_spi_slave_test <port> <freq> <addr> <size> <loop> [debug]

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 多笔数据连续发送时，如何让 CS 脚保持使能

问题现象：每包数据发送后，CS 脚都会翻转一次，重新使能，导致与外设通信异常。

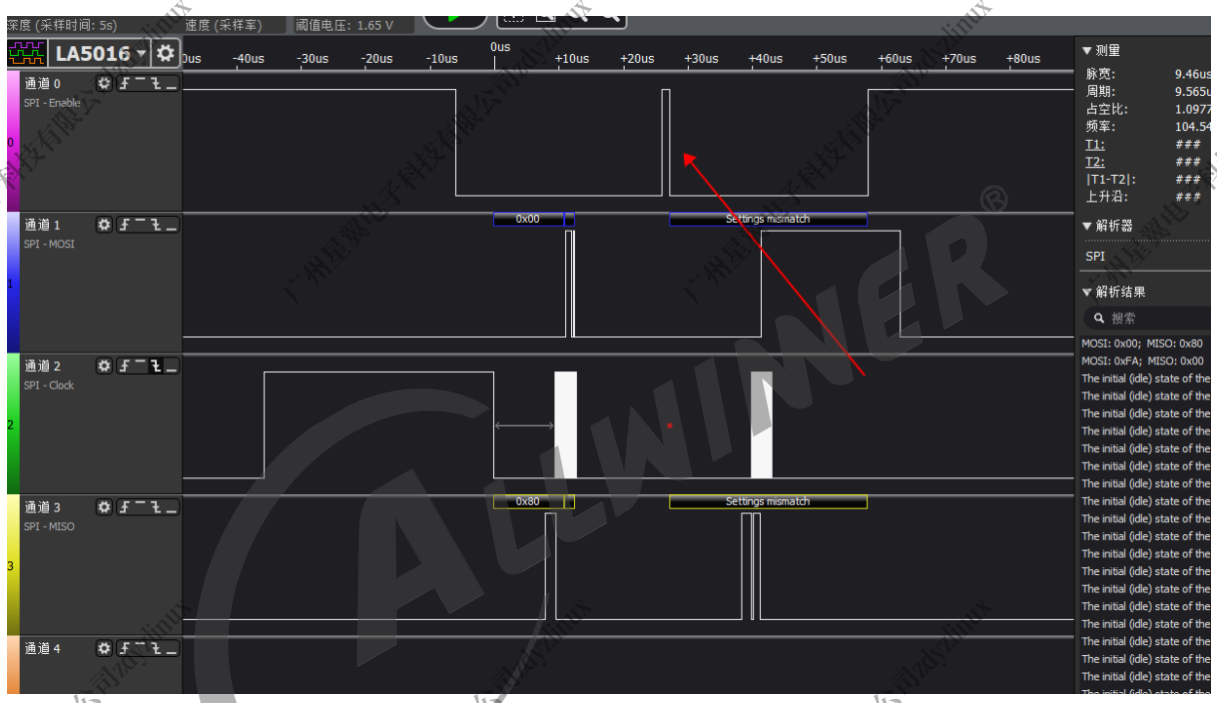

图 6-1: FAQ_1_1

解决方法：将 cs_mode 配置为手动，并将需要 CS 信号包在内的多包数据，一次性传给驱动。

```
hal_spi_master_config_t cfg = { 0 };
hal_spi_master_transfer_t tr[2];
......
cfg.cs_mode = HAL_SPI_CS_SOFT;
......
hal_spi_init(port, &cfg);
......
hal_spi_xfer(port, &tr, ARRAY_SIZE(tr));
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。